

SIMATS ENGINEERING

ASSESSMENT TOOL 2

K. Lokesh Chandra
192425358

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (High)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	5
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	5
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	5
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	5
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	5
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	5

7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	5
8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	5
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	5
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	5

Code of Conduct:

I K. Lokesh Chandra (192425358) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Lokesh Chandra

RUBRICS FOR CALCULATION:

Scenario based assessment					
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts

		address the scenario			
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. School Management System

Scenario

A school wants to maintain details of students, classes, and class teachers.

Relations

STUDENT(StudentID, StudentName, ClassID)

CLASS(ClassID, ClassName, TeacherID)

TEACHER(TeacherID, TeacherName)

SQL Query

```
SELECT S.StudentID,  
       S.StudentName,  
       C.ClassName,  
       T.TeacherName  
FROM Student S  
JOIN Class C  
ON S.ClassID = C.ClassID  
JOIN Teacher T  
ON C.TeacherID = T.TeacherID;
```

Explanation

The query joins Student, Class, and Teacher tables to display each student's class and corresponding class teacher.

2. Hospital Appointment System

Relational Schema

PATIENT(PatientID, PatientName)

DOCTOR(DoctorID, DoctorName, Specialization)

APPOINTMENT(AppointmentID, PatientID, DoctorID, AppointmentDate)

SQL Query

```
SELECT D.DoctorID,  
       D.DoctorName,  
       COUNT(A.AppointmentID) AS TotalAppointments  
FROM Doctor D  
JOIN Appointment A  
ON D.DoctorID = A.DoctorID  
WHERE MONTH(A.AppointmentDate)=MONTH(CURDATE())  
AND YEAR(A.AppointmentDate)=YEAR(CURDATE())  
GROUP BY D.DoctorID,D.DoctorName  
HAVING COUNT(A.AppointmentID)>10;
```

Explanation

The query counts appointments of each doctor during the current month and displays only doctors with more than 10 appointments.

3. E-Commerce System

SQL Query

```
SELECT ProductID,  
       ProductName  
FROM Product  
WHERE ProductID NOT IN  
(  
  SELECT ProductID  
  FROM OrderDetails  
);
```

Explanation

The subquery retrieves ordered products, while the main query displays products that have never been ordered.

4. University Database

Relations

Student(StudentID, Name)
Enrollment(StudentID, CourseID)
Course(CourseID, CourseName)

Relational Algebra

$$\text{DBMS} \leftarrow \pi_{\text{StudentID}}(\sigma_{\text{CourseName}='DBMS'}(\text{Enrollment} \bowtie \text{Course}))$$
$$\text{OS} \leftarrow \pi_{\text{StudentID}}(\sigma_{\text{CourseName}='OS'}(\text{Enrollment} \bowtie \text{Course}))$$
$$\text{Result} \leftarrow \text{DBMS} \cap \text{OS}$$

Explanation

The intersection operator returns students enrolled in both DBMS and Operating Systems.

5. Payroll System

View

```
CREATE VIEW LowSalaryEmployees AS  
SELECT E.EmpID,  
       E.EmpName,  
       E.DeptID,  
       E.Salary  
FROM Employee E
```

```
WHERE Salary <
(
SELECT AVG(Salary)
FROM Employee E2
WHERE E.DeptID=E2.DeptID
);
```

Update Query

```
UPDATE Employee
SET Salary = Salary * 1.10
WHERE EmpID IN
(
SELECT EmpID
FROM LowSalaryEmployees
);
```

Explanation

The view identifies employees earning below their department's average salary. The update increases their salary by 10%.

6. Banking System

Tables

Customer

Account

Transaction

Branch

SQL Query

```
SELECT C.CustomerName,
       A.AccountNumber,
       T.TransactionID,
       T.Amount,
       B.BranchName
FROM Customer C
JOIN Account A
ON C.CustomerID=A.CustomerID
JOIN Transaction T
ON A.AccountNumber=T.AccountNumber
JOIN Branch B
ON A.BranchID=B.BranchID;
```

Explanation

JOIN operations combine customer, account, transaction, and branch information into one result.

7. Retail Store

SQL Query

```
SELECT Category,  
       SUM(SalesAmount) AS TotalSales  
FROM Sales  
GROUP BY Category  
HAVING SUM(SalesAmount)>10000;
```

Explanation

GROUP BY calculates total sales for each category, while HAVING filters categories with sales exceeding ₹10,000.

8. Library System

Relations

Member(MemberID, Name)

Book(BookID, Genre)

Borrow(MemberID, BookID)

Tuple Relational Calculus

$$\{$$
$$M.Name \mid$$
$$Member(M)$$
$$\wedge$$
$$\forall G$$
$$(\$$
$$Genre(G)$$
$$\rightarrow$$
$$\exists B$$
$$(\$$
$$Book(B)$$
$$\wedge$$
$$B.Genre=G$$
$$\wedge$$
$$Borrow(M,B)$$
$$)$$
$$)$$
$$\}$$

Explanation

The expression retrieves members who have borrowed books from every available genre.

9. Inventory Management

Create Tables

```
CREATE TABLE Category
(
  CategoryID INT PRIMARY KEY,
  CategoryName VARCHAR(50)
);

CREATE TABLE Product
(
  ProductID INT PRIMARY KEY,
  ProductName VARCHAR(50),
  Price DECIMAL(10,2)
  CHECK(Price>0),
  CategoryID INT,
  FOREIGN KEY(CategoryID)
  REFERENCES Category(CategoryID)
);
```

Enforcement

Valid Insert

```
INSERT INTO Category
VALUES(1,'Electronics');
```

```
INSERT INTO Product
VALUES(101,'Laptop',50000,1);
```

Invalid Insert

```
INSERT INTO Product
VALUES(102,'Mouse',500,-1);
```

Explanation

The CHECK constraint ensures the price is positive, while the FOREIGN KEY allows only valid Category IDs.

10. HR System

Relational Schema

```
Employee(EmpID, EmpName, DeptID)
Attendance(EmpID, AttendanceDate, Status)
Performance(EmpID, Month, Rating)
```

Materialized View

```
CREATE MATERIALIZED VIEW MonthlyAttendance
AS
SELECT EmpID,
```



```
COUNT(*) AS DaysPresent
FROM Attendance
WHERE Status='Present'
GROUP BY EmpID;
CREATE MATERIALIZED VIEW PerformanceSummary
AS
SELECT EmpID,
AVG(Rating) AS AverageRating
FROM Performance
GROUP BY EmpID;
```

Explanation

The materialized views store precomputed attendance and performance summaries, improving reporting speed.